

|         |                                                   |
|---------|---------------------------------------------------|
| EX.NO:1 | <b>PRINT PRIME NUMBERS UPTO GIVEN<br/>INTEGER</b> |
| DATE:   |                                                   |

**AIM:**

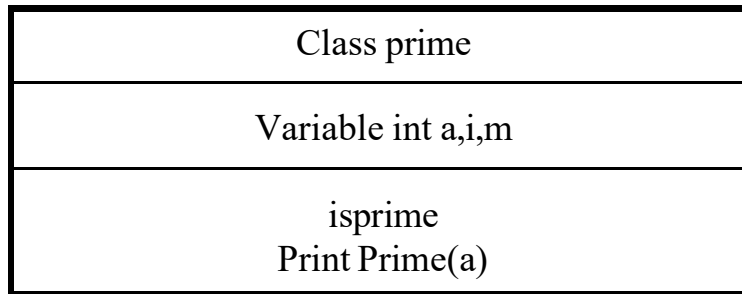
To write a java program that prompts the user for an integer and then count out all prime number upto that interger.

**ALGORITHM:**

1. Start
2. Importing the scanner class from Java.util package,which is used for reading user input .
3. In main method(main)
  - a. It creates a scanner object named scanner to read user input from reading user from the console.
  - b. It prompts the user to enter an interger.
  - c. It read the integer entered an by the user and stores it in the variable “user input”.
  - d. It close the scanner object to release resources.
  - e. It print a message indurating that it print prime number upto entered integer .
  - f. It calls the print primes function passing the user input as an argument.
4. In is prime function
  - a. It takes an integer 'num' as input and return true if the number is prime otherwise false.
  - b. If first check if the checks if the number is less than or equal to 1,in which case it return false because prime number are greater 1
  - c. It then iterates from 2 to the square root of num ,checking if num is divisible by any number in that range.If it is divisible by any number it return false,indicating that the number is not prime.
  - d. It the loop completes without finding any divisors,it return true,indicating that number is prime.
5. Print prime function
  - a. If takes an integer limit as input and prints all prime number upto to that limit .
  - b. It iterates from 2 to the limit .
  - c. For each number i ,it calls the isprime function to check if isprime
6. Stop.

## **FLOWCHART:**

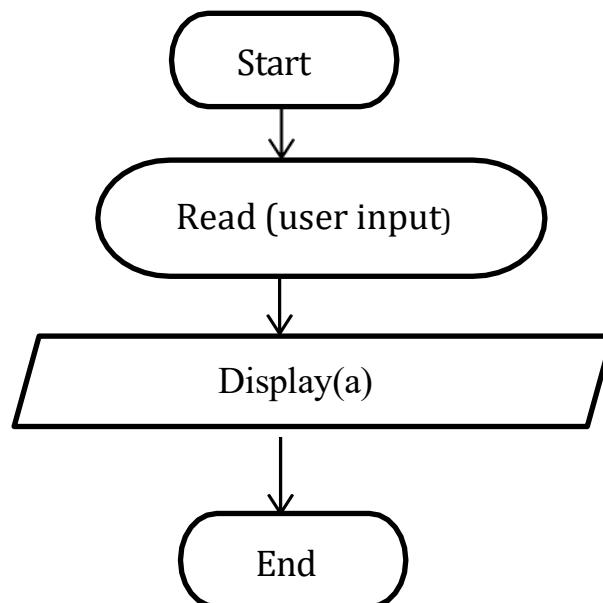
### **CLASS DIAGRAM**



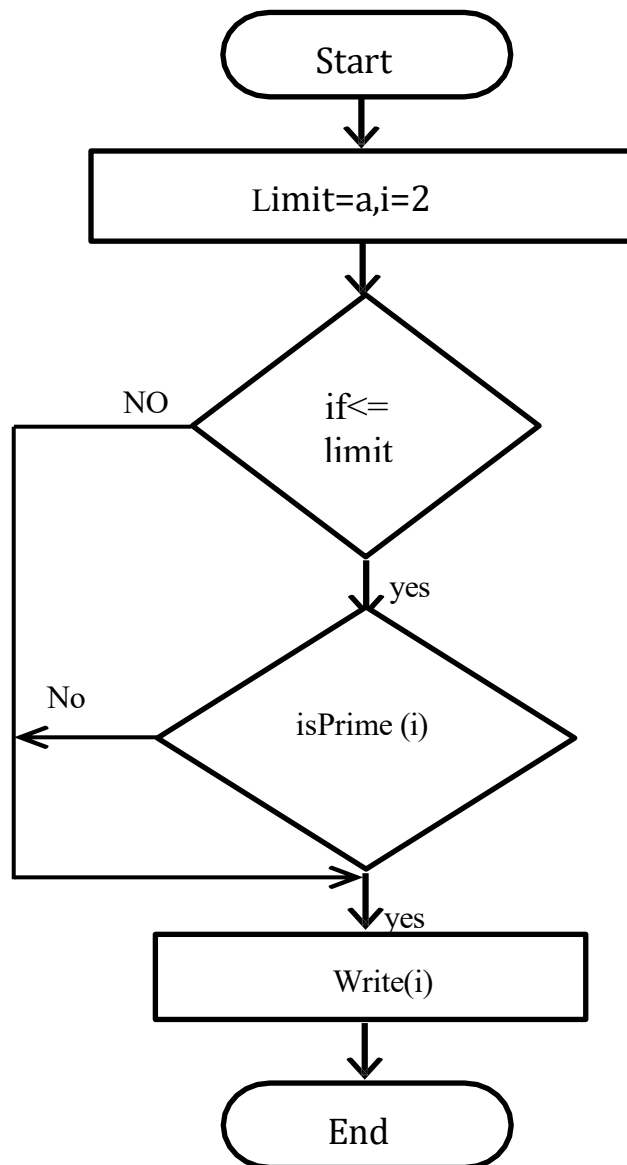
## **FLOWCHART:**

Main ();

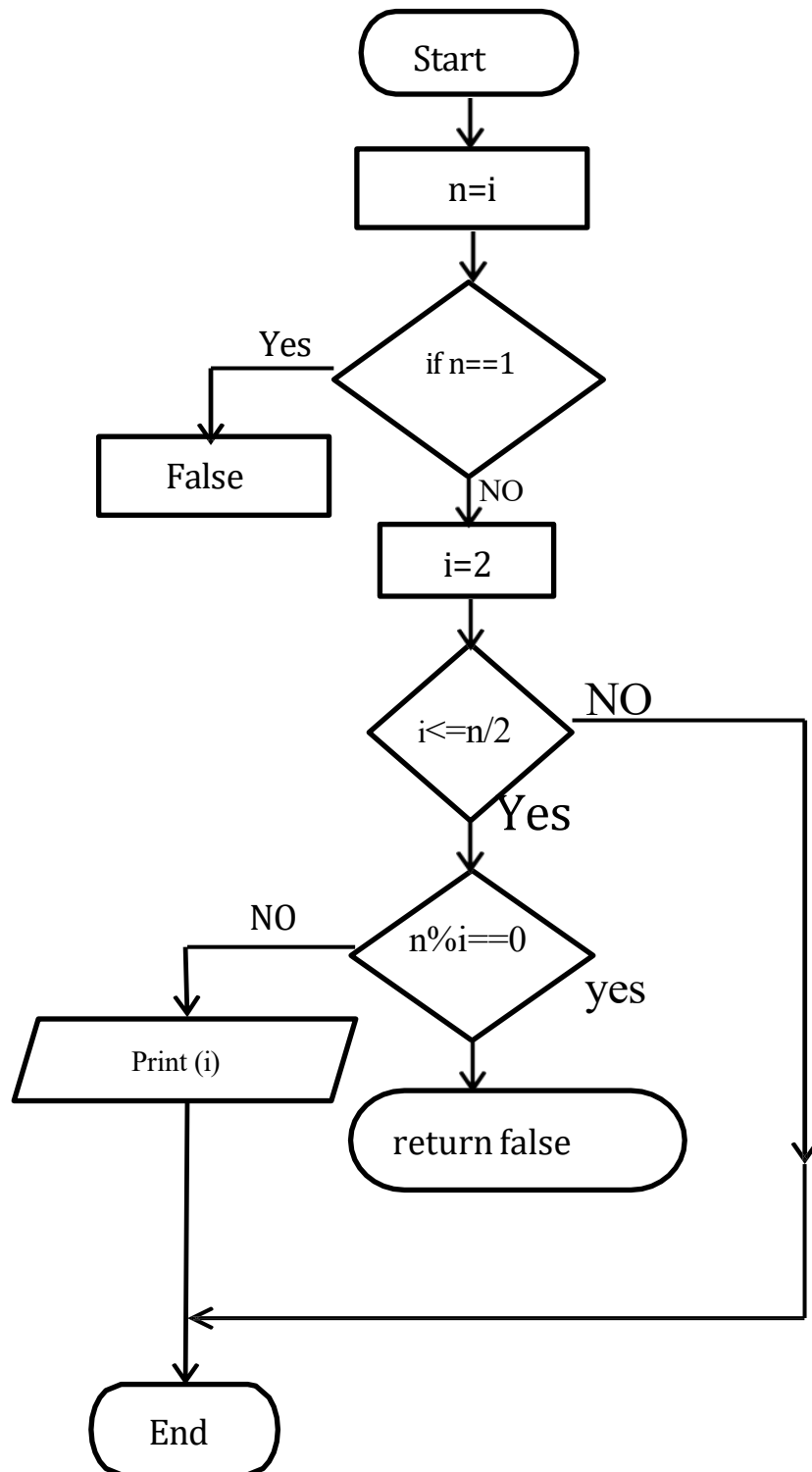
**Print Prime number**



## Display (limit)



**isPrime()**



## **PROGRAM:**

```
import java.util.*;
public class Prime {
    Public Static void main(String []args){
        Scanner s = new Scanner(System.in);
        System.out.print("Enter an integer: ");
        int a = s.nextInt();
        s.close();
        System.out.println("Prime numbers up to " + a + ":");
        display(a);
    }
    static Boolean isPrime(int n) {
        if (n <= 1) {
            return false;
        }
        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0) {
                return false;
            }
        }
        return true;
    }
    static void display(int limit) {
        for (int i = 2; i <= limit; i++) {
            if (isPrime(i)) {
                System.out.print(i + " ");
            }
        }
    }
}
```

## **OUTPUT**

Enter interger:10

Prime number

upto10:

2 3 5 7

## **RESULT:**

Thus the program has been executed successfully.

|                |                             |
|----------------|-----------------------------|
| <b>EX.NO:2</b> | <b>MATRIX MULIPLICATION</b> |
| <b>DATE:</b>   |                             |

**AIM:**

To write a java program to multiply two given matrices .

**ALGORITHM:**

1 Start

2 Import the scanner class from java.util package for user input.

3 In main method

- a. It creates a scanner object named scanner to read user input from the console.
- b. It prompts the user to enter the dimension of the frist matrix (no of rows and close).
- c. It reads the dimension entered by the user and stones them in variable rows 7and colum1.
- d. It call input matrix function passing rows 1 and columns 1 argument to input the frist matrix.
- e. If prompts the user to entered the dimension the second matrix(number of rows and columns)
- f. If read the dimension enter by the user and stores them in variable rows 2 and columns 2.
- g. It calls input matrix function passing rows 2 as argument to input the second matrix.
- h. If check if matrix muplication is possible by comparing the number of columns of the frist matrix with the row of the second matrix .
- i. If muplication is possible ,it calls the multiply matrix function passing the two matrices as arguments and Store the result .
- j. It display the resultant matrix if multiplication is possible otherwise,if prints a message indicating that matrix mulplication is not possible
- k. It closes the scanner object to avoid resource leak.

4 input matrix function

- a. It takes the number of rows and columns as input.
- b. It prompts the user to enter the elements of the matrix row of row .
- c. It reads the elements entered by the user and stores then in the matrix .

d. It return the matrix.

5. In multiply matrices function

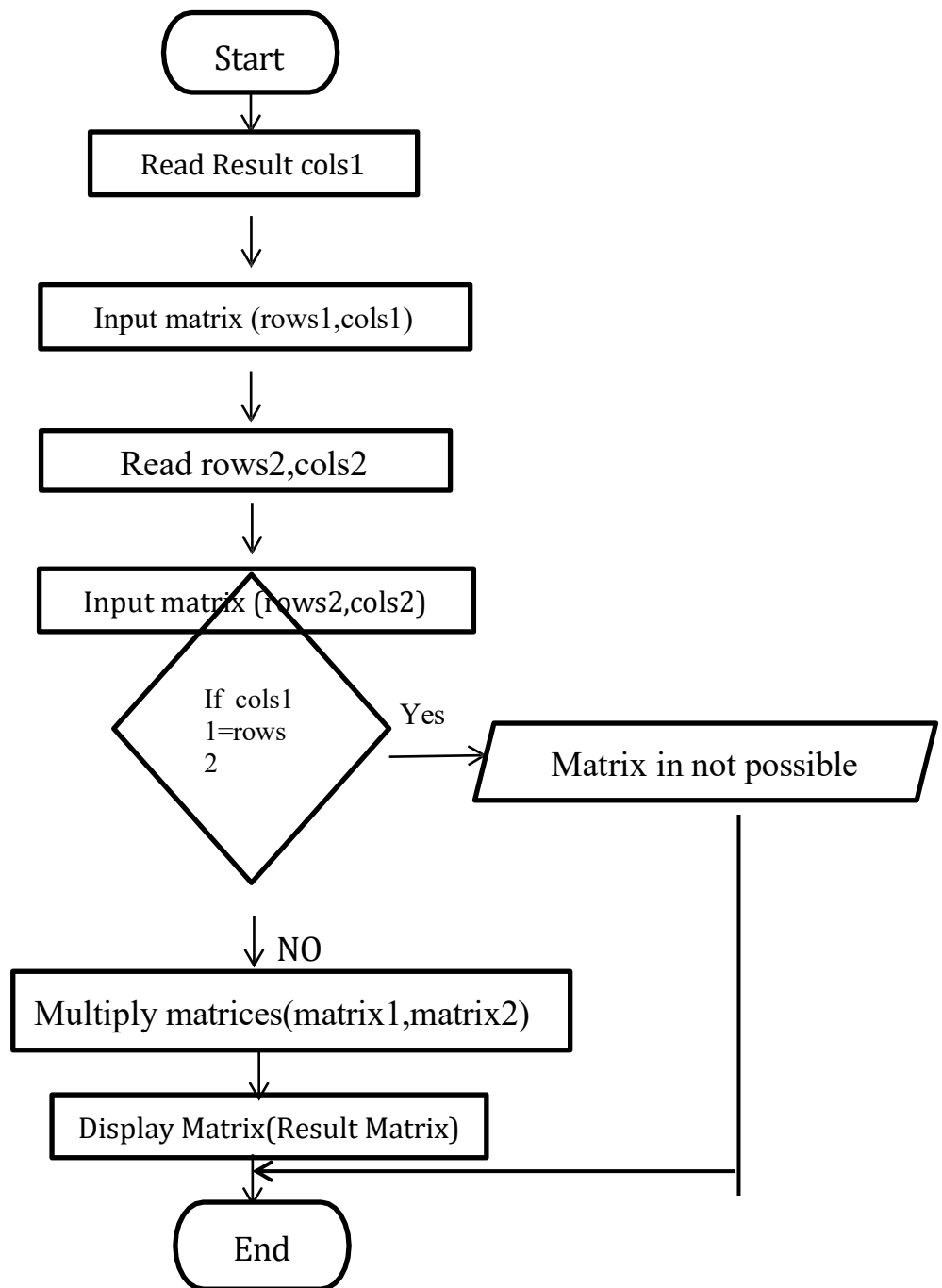
a. This function takes two matrices as input.

b. If initializes a result matrix with dimension based on the number of rows of the frist matrix and the number of columns of the second matrix .

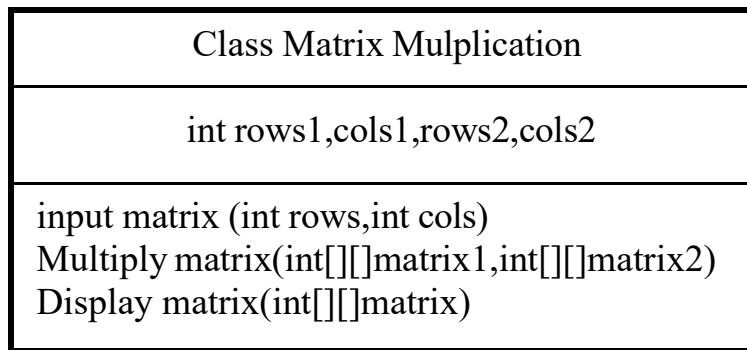
6. In display matrix function ,it iterates over the rows of the elements of each row ,printing them with space between elements.

7. stop

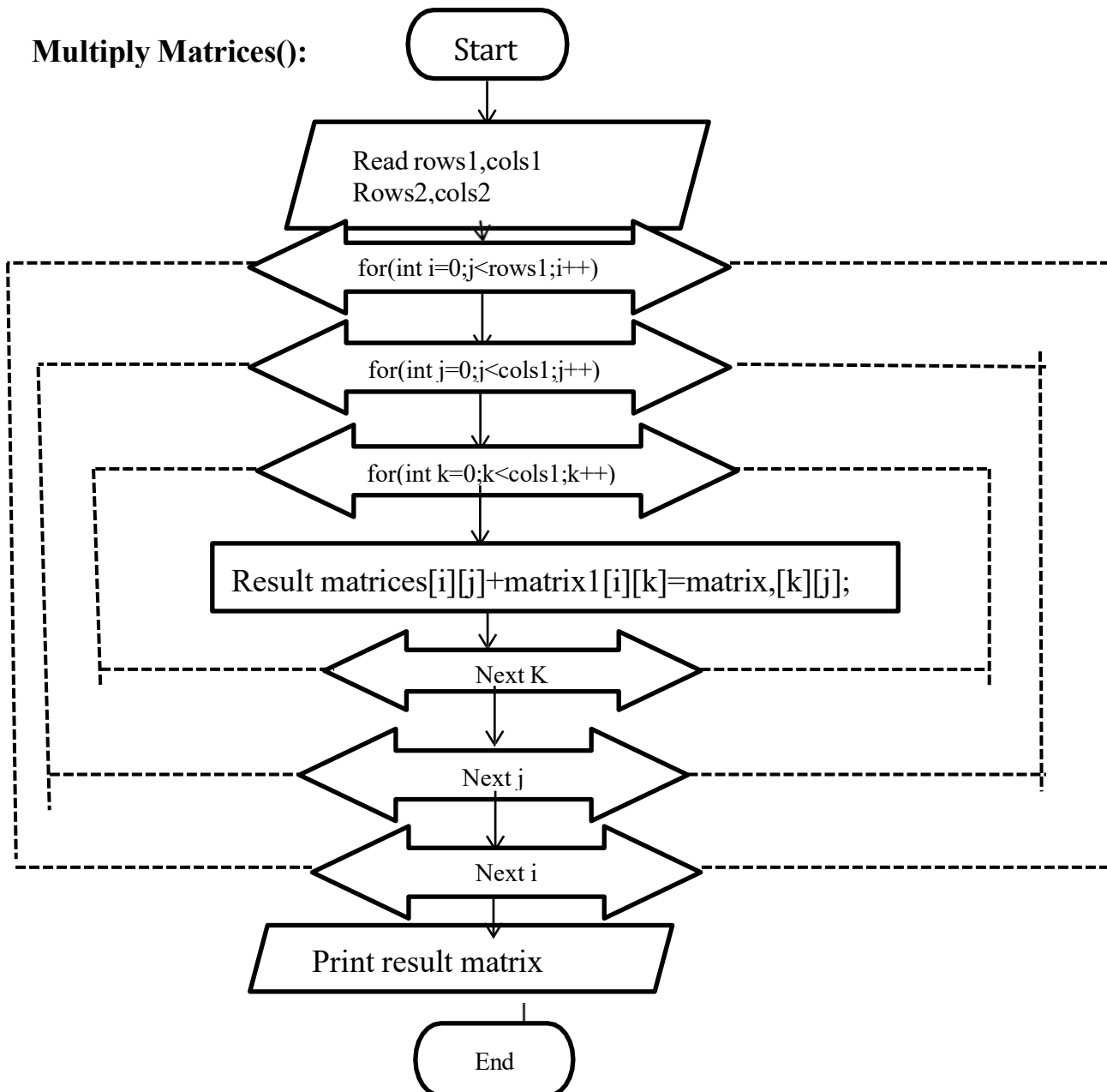
**FLOWCHART:**

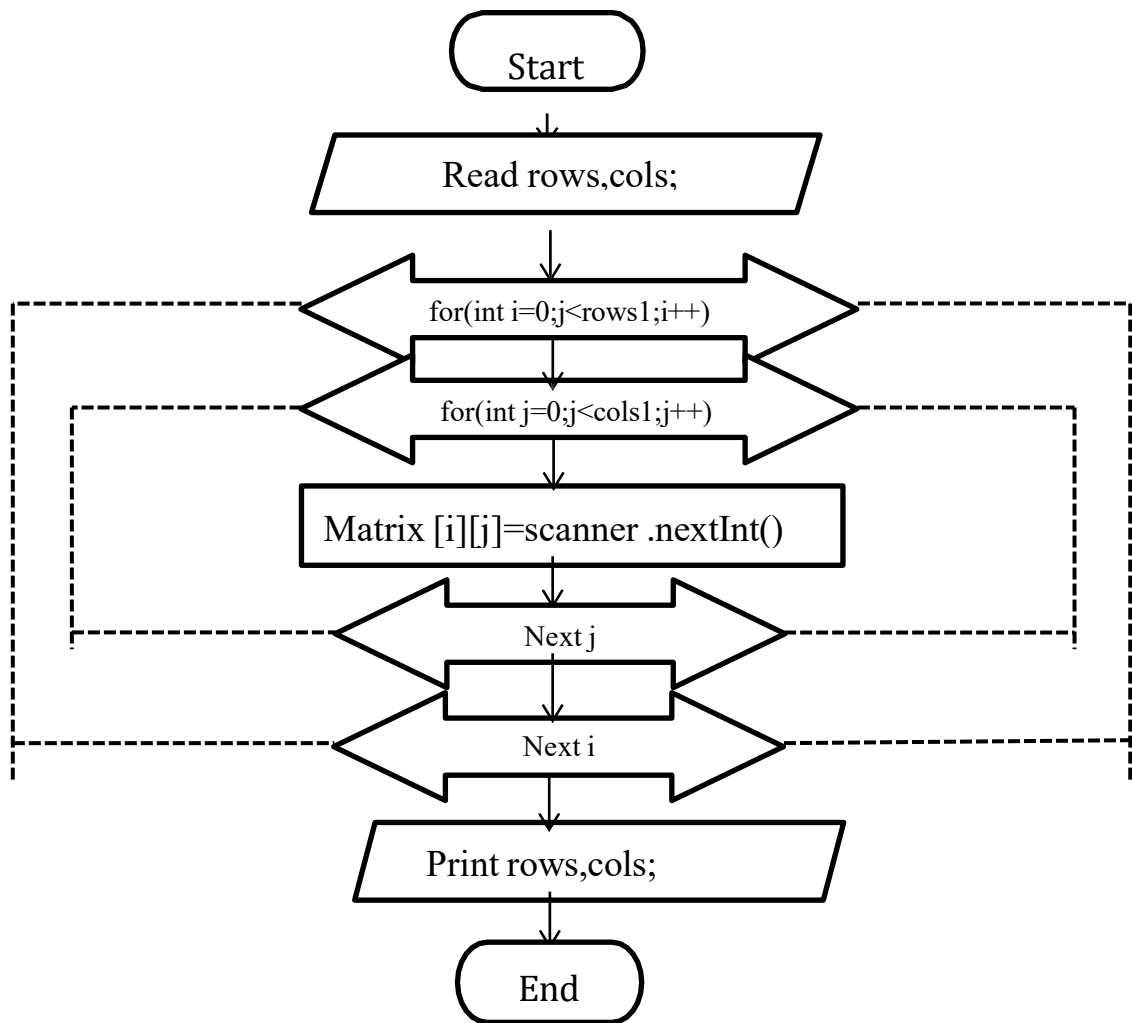


## CLASS DIAGRAM



**Multiply Matrices():**





## **PROGRAM:**

```
import java.util.Scanner;

class MatrixMul {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int a[][] = new int[2][2];
        int b[][] = new int[2][2];
        int c[][] = new int[2][2];
        System.out.println("Enter Matrix A:");
        for (int i = 0; i < 2; i++)
            for (int j = 0; j < 2; j++)
                a[i][j] = sc.nextInt();
        System.out.println("Enter Matrix B:");
        for (int i = 0; i < 2; i++)
            for (int j = 0; j < 2; j++)
                b[i][j] = sc.nextInt();
        for (int i = 0; i < 2; i++)
            for (int j = 0; j < 2; j++)
                for (int k = 0; k < 2; k++)
                    c[i][j] += a[i][k] * b[k][j];
        System.out.println("Result:");
        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 2; j++)
                System.out.print(c[i][j] + " ");
            System.out.println();
        }
    }
}
```

## **OUTPUT:**

Enter the dimensions of the first matrix:

Number of rows: 2

Number of columns: 2

Enter the elements of the matrix:

3

4

5

8

Enter the dimensions of the second matrix:

Number of rows: 2

Number of columns: 2

Enter the elements of the matrix:

1

1

2

2

Resultant matrix after multiplication:

11 11

21 21

## **RESULT:**

Thus the program has been executed successfully.

|         |                                                                       |
|---------|-----------------------------------------------------------------------|
| EX.NO:3 | <b>DISPLAY THE NUMBER OF CHARACTER, LINES<br/>AND WORDS IN A TEXT</b> |
| DATE:   |                                                                       |

### **AIM:**

To write java program that display the number the number of characters lines and words in a text.

### **ALGORITHM:**

Step1:Start.

Step2:Importing the scanner class from java.util package  
which is used for reading user input.

Step3:Define the text analyzer class named text analyzes.

Step4:Implement the main method.

- a. Inside the main method.
- b. Inside the scanner object to take user input.
- c. Display a prompt asking the user to enter some text.
- d. Read the input text from the user using scanner .nextLine() and store it in the variable input text.
- e. close the scanner to prevent resource leak.

Step5:Implement the count characters method:

- a. Create a static method named count charactes that takes a string text as a parameter.
- b. Return the length of the input text using text.length().

Step 6:Implement the countless method:

- a. Create a stasic method named countless that makes takes a string text as a parameter.
- b. Split the input text into an array of lines using the regular expression `\r||n||r||n\`
- c. Return the length of the resulting array.

Step.7: Implement the count words method:

- a. Create a static method name count the word that take as a string text as a parameter.
- b. split the input text into an array of words using the regular expression `"3+(matchas and or more white-space charlatters)`

- c. Return the length of the resulting array.

Step 8:display the analysis result:

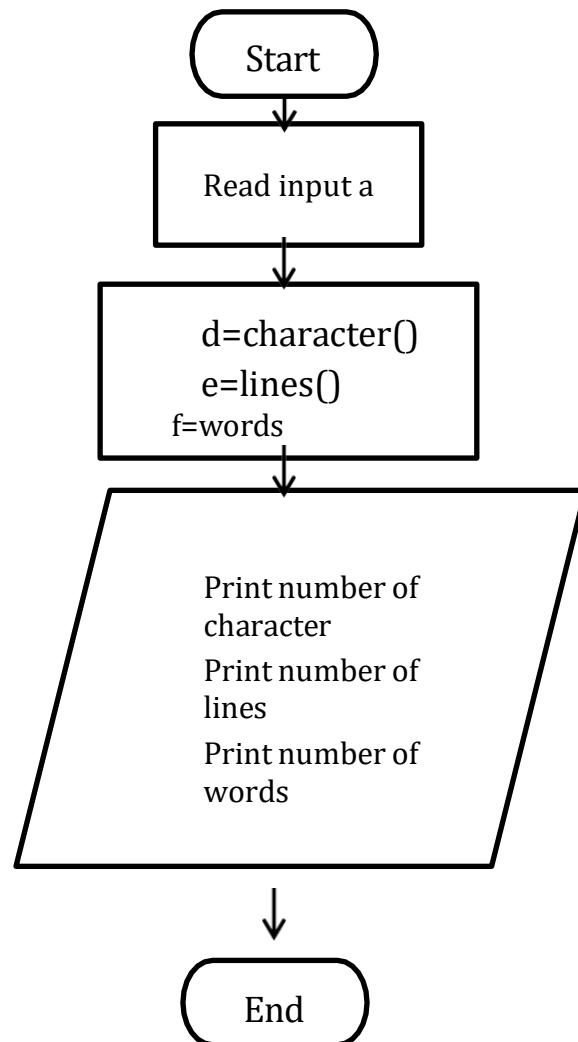
- a. a)print the analysis result to console in the main method.
- b. display the number of character words and lines

step 9:stop

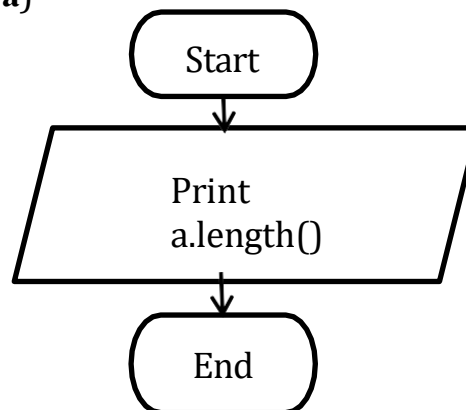
**CLASS DIAGRAM:**

|                                               |
|-----------------------------------------------|
| intClass Text                                 |
| int a,b,c,d,e,f                               |
| int character()<br>int lines()<br>int words() |

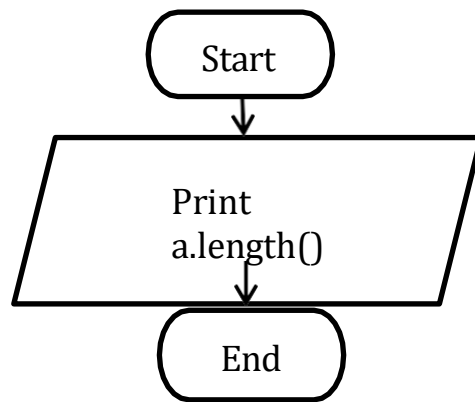
## FLOW CHART



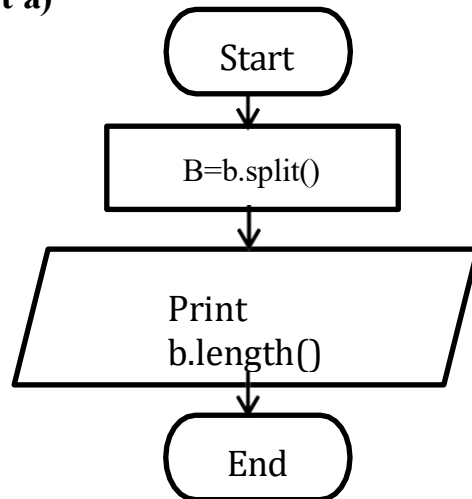
### **Characters(input a)**



### Words(input a)



### Lines(input a)



## **PROGRAM :**

```
import java.util.Scanner;
public class Text {
    static int characters(String a) {
        return a.length();
    }
    static int Lines(String a) {
        String[] lines = a.split("\r\n\r\n");
        return lines.length;
    }
    static int Words(String a) {
        String[] Words = a.split("\\s+");
        return Words.length;
    }
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.println("Enter some text:"); String
        inputText = s.nextLine();
        int d = characters(inputText);
        int e = Lines(inputText);
        int f = Words(inputText);
        System.out.println("\nAnalysis Result:");
        System.out.println("Number of characters: " + d);
        System.out.println("Number of lines: " + e);
        System.out.println("Number of words: " + f);
    }
}
```

## **OUTPUT:**

Enter some text:

java programing

Analysis Result:

Number of characters: 15

Number of lines: 1

Number of words: 2

## **RESULT:**

Thus the program has been executed successfully.

|                |                                                   |
|----------------|---------------------------------------------------|
| <b>EX.NO:4</b> | <b>GENERATE RANDOM NUMBERS USING RANDOM CLASS</b> |
| <b>DATE:</b>   |                                                   |

### **AIM:**

To write a java program to generate random number between two given limit using random class

### **ALGORITHM:**

Step1:Start

Step 2:Define the class named Random Generator.

Step 3:Declare variables lowerlimit and upperlimit to set the range for random number generation.

Step 4: Create a random object as an instance of the random class to generate random numbers.

Step 5: Generate a random number within the specified range.

a) use the nextInt method of the random class to generate a random integer within the specified range (upper limit and lower limit) and add lower limit to ensure it falls within the desired range.

b) Store the generated random number in the variable ran Num

Step 6 : print generated random number

Step7: Check the range of the generated number and print message accordingly.

a) use conditional statement (if, else, if and else )to check the range of generated number

b) Print message based on the condition

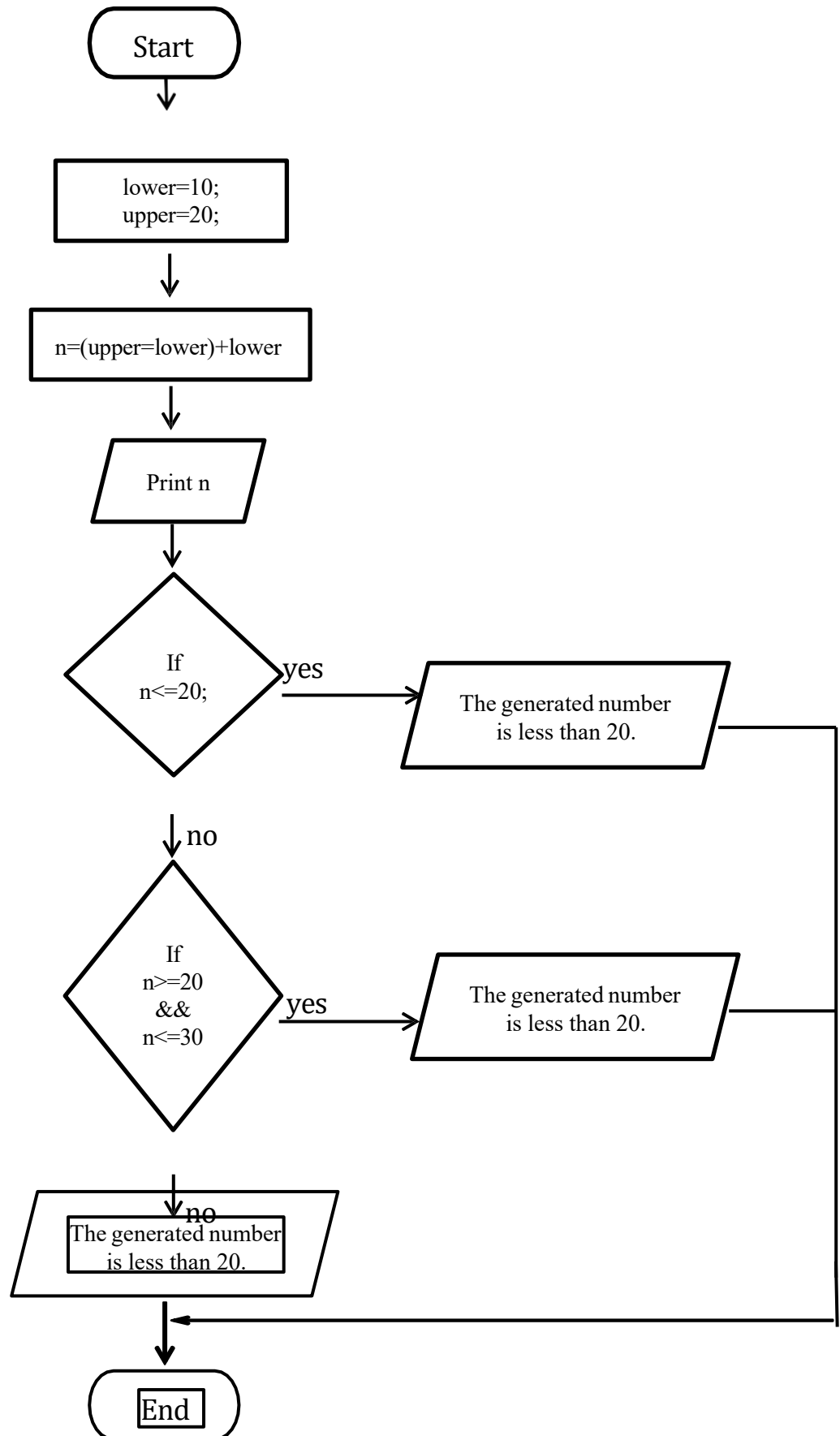
i) If the number is less than 20

If the number is between 20 and 30(inclusive).

ii) If the number is greater than 30

Step 8: Stop.

## FLOW CHART



### **PROGRAM:**

```
import java.util.Random;
public class RanNumGenerator {
    public static void main(String[] args) {
        int lower = 10;
        int upper = 50;
        Random r = new Random();
        int n = r.nextInt(upper - lower + 1) + lower;
        System.out.println("Generated Random Number: " + n);
        if (n < 20) {
            System.out.println("The generated number is less
            }
        else if (n >= 20 && n <= 30) {
            System.out.println("The generated number is between 20 and
            30(inclusive).");
        }
        else {
            System.out.println("The generated number is greater than 30.");
        }
    }
}
```

### **OUTPUT:**

Generated Random Number: 39  
The generated number is greater than 30.

### **RESULT:**

Thus the program has been executed successfully .

|                |                                                  |
|----------------|--------------------------------------------------|
| <b>EX.NO:5</b> | <b>STRING MANIPULATION USING CHARACTER ARRAY</b> |
| <b>DATE:</b>   |                                                  |

**AIM:**

To perform string manipulating using character array.

**ALGORITHM:**

Step1:Start

Step2:Declare and initialize two character array str 1and str2 with the value “Hello” and “world”,respectively.

Step3:String length:

- use the length property of the character arrays(str.1 length and str 2 length) to find the length of each string.
- Print the length of both string.

Step4:character at a particular position:

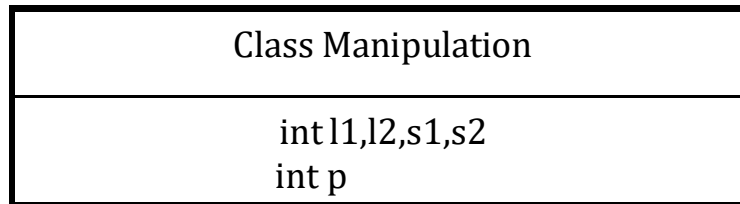
- Specify a position(0-base index )as position=2.
- Access the character at the specified position in str.1 using array indexing and store it in the variable char at specified
- Print the character at specified position in string 1.

Step 5: concatenating two string.

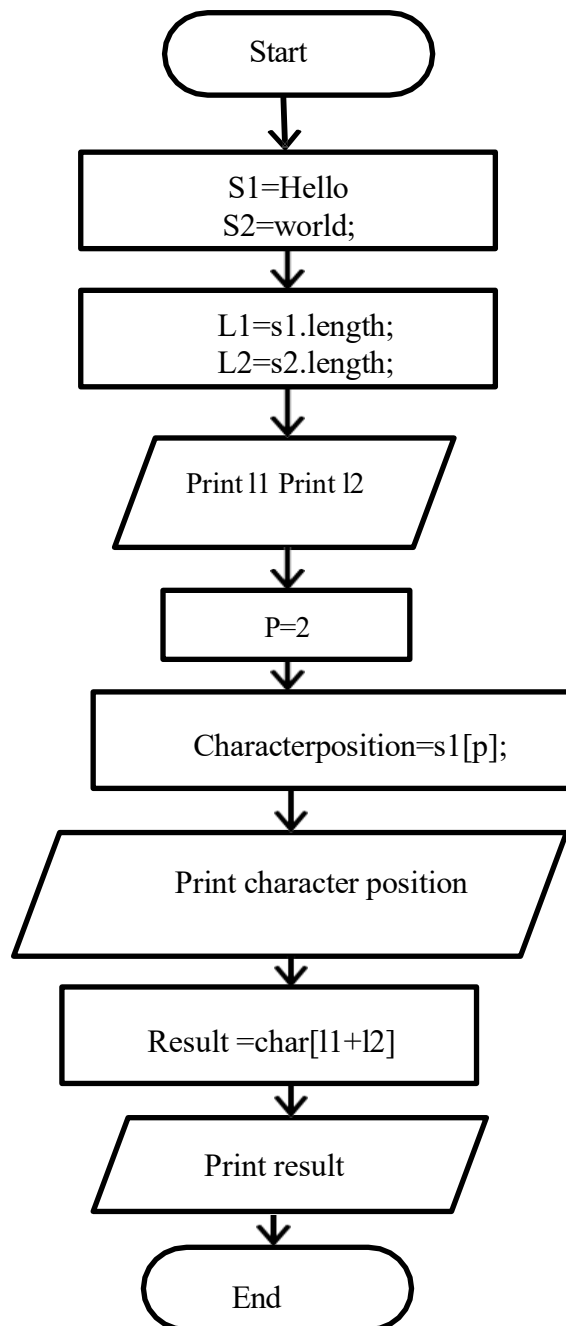
- Create a new character array result with a length equal to the length str1 and str2.
- System array copy to copy the content of str1 to the beginning of result.
- Use system array.copy again to concatenate the content of str 2 to the end of result .
- Convert the resulting character array result to a string using new string (result).
- Print the concatenated string.

Step 6:Stop.

### CLASS DIAGRAM:



### FLOW CHART:



## **PROGRAM:**

```
public class StringManipulation {
    public static void main(String[] args) {
        System.out.println("STRING MANIPULATION");
        System.out.println("\n*****");
        char[] s1 = "hello".toCharArray();
        char[] s2 = "World".toCharArray();
        int l1 = s1.length;
        int l2 = s2.length;
        System.out.println("String1 length: " + l1);
        System.out.println("String2 length: " + l2);
        int p = 2;
        char a = s1[p];
        System.out.println("Character at position " + p + " in String 1: " + a);
        char[] result = new char[l1 + l2];
        System.arraycopy(s1, 0, result, 0, l1);
        System.arraycopy(s2, 0, result, l1, l2);
        System.out.println("Concatenated string: " + new String(result));
    }
}
```

## **OUTPUT:**

### **STRING MANIPULATION**

\*\*\*\*\*

String1 length: 5

String2 length: 5

Character at position 2 in String 1: l

Concatenated string: helloWorld

## **RESULT:**

Thus the program has been executed successfully.

|                |                                       |
|----------------|---------------------------------------|
| <b>EX.NO:6</b> | <b>STRING OPERATIONS STRING CLASS</b> |
| <b>DATE:</b>   |                                       |

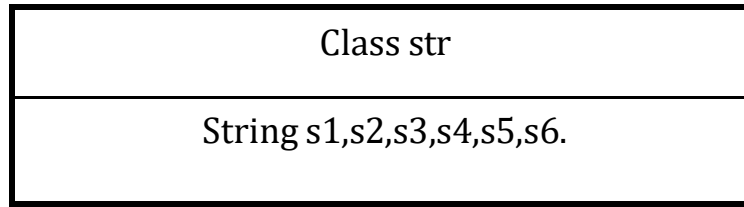
**AIM:**

To write a java program to perform string operation using string class

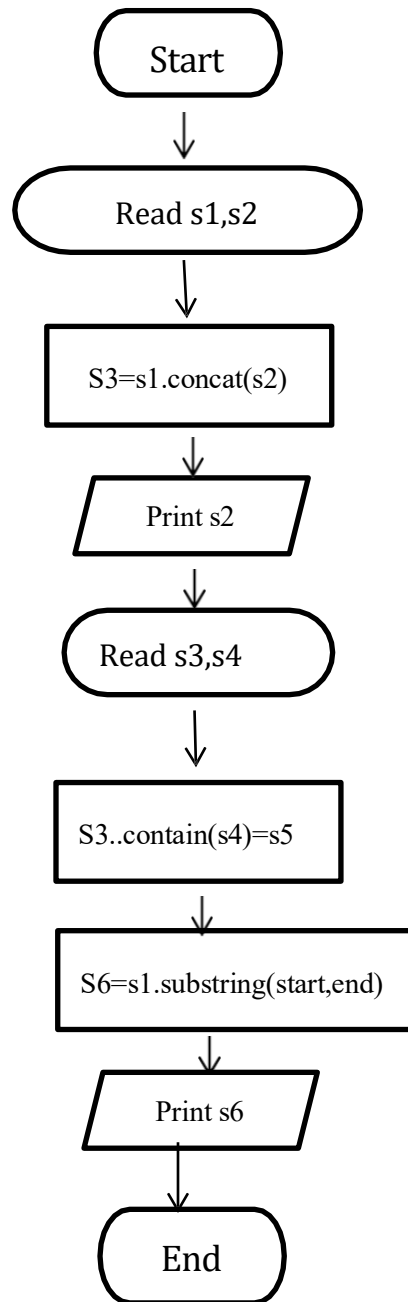
**ALGORITHM:**

1. Start
2. Input a string from the user
3. Display the input string and its length
4. Input a substring to search
5. Check if the input string contains the substring
6. Display the result (found or not found)
7. Input start and end indices
8. Extract a substring from the input string using the start and end indices
9. Display the extracted substring
10. Stop

**CLASS DIAGRAM:**



**FLOWCHART:**



## **PROGRAM:**

```
import java.util.*;
public class str {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.println("Enter the first string:");
        String s1 = s.nextLine();
        System.out.println("Enter the second string:");
        String s2 = s.nextLine();
        String s3 = s1.concat(s2);
        System.out.println("Concatenated String: " + s3);
        System.out.println("Main string: " + s1);
        System.out.println("Enter the substring:");
        String s4 = s.nextLine();
        boolean s5 = s3.contains(s4);
        System.out.println("Substring found? " + s5);
        System.out.println("Main string for substring extraction: " + s1);
        System.out.println("Enter the starting index:");
        int start = s.nextInt();
        System.out.println("Enter the ending index:");
        int end = s.nextInt();
        String s6 = s1.substring(start, end);
        System.out.println("Extracted Substring: " + s6);
        s.close();
    }
}
```

## **OUTPUT:**

Enter the first string: java  
Enter the second string: programming  
Concatenated String: javaprogramming  
Main string: java  
Enter the substring: av  
  
Substring found? true  
Main string for substring extraction: java  
Enter the starting index:  
1  
Enter the ending index:  
4  
Extracted Substring: ava

## **RESULT :**

Thus the program has been executed successfully.

|                |                                              |
|----------------|----------------------------------------------|
| <b>EX.NO:7</b> | <b>STRING OPERATIONS USING STRING BUFFER</b> |
| <b>DATE:</b>   |                                              |

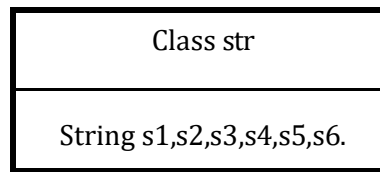
**AIM:**

To write a java program using operation using string buffer class.

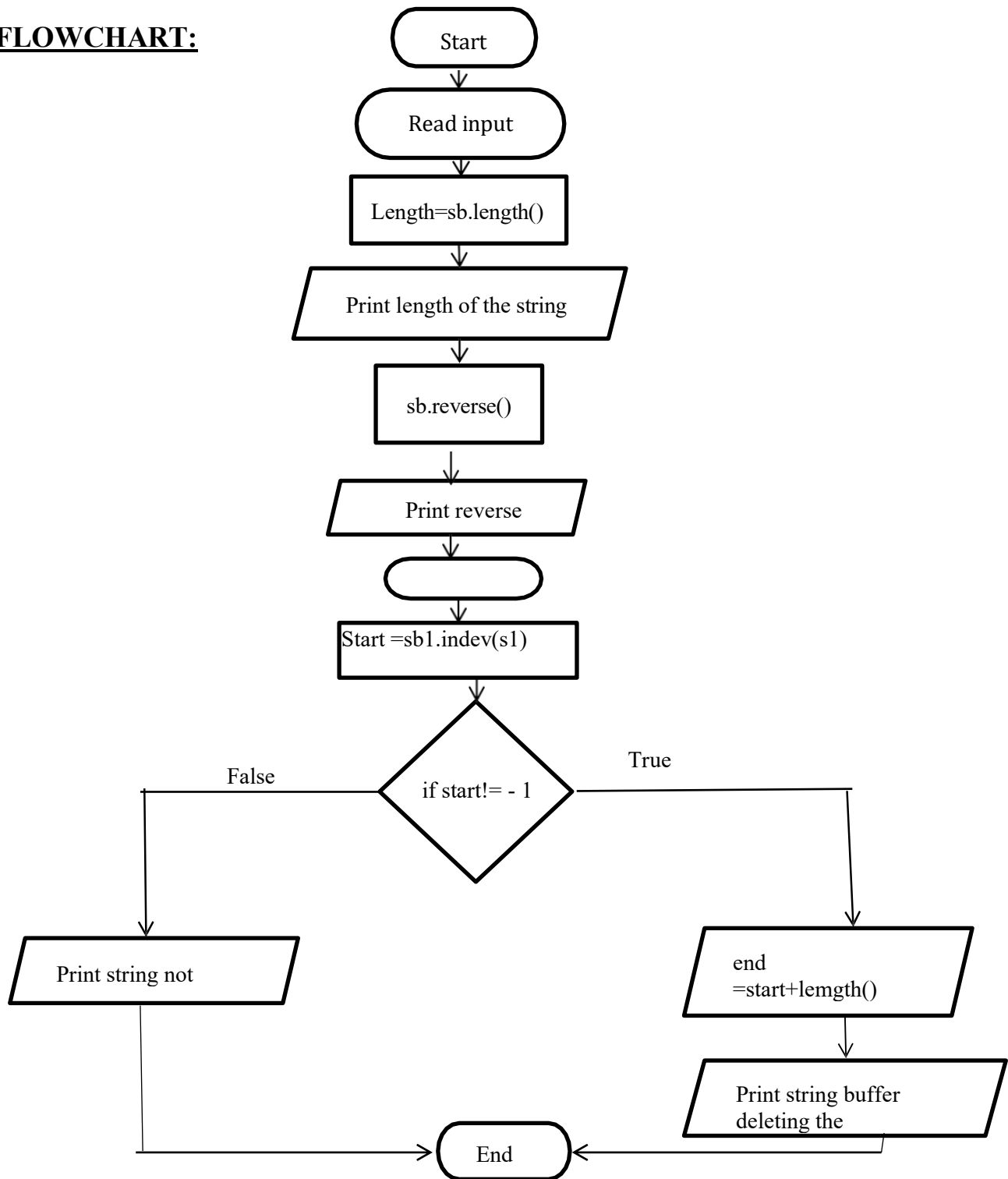
**ALGORITHM:**

1. Start program
2. Create Scanner for input
3. Read string from user and create StringBuffer object
4. Display length using length() method
5. Reverse string using reverse() method and display
6. Reverse back to original (sb.reverse())
7. Read start and end indices for deletion
8. Delete characters from start to end-1 using delete(start, end)
9. Display modified string
10. Close Scanner
11. End

### CLASS DIAGRAM:



### FLOWCHART:



## **PROGRAM:**

```
import java.util.Scanner;
public class buffer {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.println("Enter a String: ");
        String input = s.nextLine();
        StringBuffer sb = new StringBuffer(input);
        int length = sb.length();
        System.out.println("Length of the string: " + length);
        sb.reverse();
        System.out.println("Reversed string: " + sb);
        sb = new StringBuffer(input);
        System.out.print("Enter a substring to delete: ");
        String s1 = s.nextLine();
        int start = sb.indexOf(s1);
        if (start != -1) {
            int end = start + s1.length();
            sb.delete(start, end);
            System.out.println("String after deleting the substring: " + sb);
        } else {
            System.out.println("Substring not found in the given string.");
        }
        s.close();
    }
}
```

### **OUTPUT:**

Enter a String: welcome  
Length of the string: 7  
Reversed string: emoclew  
Enter a substring to delete: wel  
String after deleting the substring: come  
Enter a String:  
i can't  
Length of the string: 7  
Reversed string: t'nac i  
Enter a substring to delete: 't  
String after deleting the substring: i can

### **RESULT:**

Thus the program has been executed successfully .

